计算机系统概论 · 笔记（第二、三讲）

涵盖课件

1. **2.1 整数** —— 数的机器表示、补码、运算与溢出、移位与除法
2. **2.2 浮点数** —— IEEE 754 标准、舍入、C 语言中的浮点数
3. **3.1 80x86 汇编语言与 C 语言（一）** —— 机器级编程基础、数据传送与算逻指令

课件原件见本目录 `课件/` 子文件夹。

2.1 整数

预备知识：数量级与单位

$1K = 2^{10} = 1024$（kilo），$1M = 2^{20}$（Mega），$1G = 2^{30}$（Giga），$1T = 2^{40}$（Tera），$1P = 2^{50}$（Peta），$1E = 2^{60}$（Exa）。

- 1 个二进制位：**bit**（比特）
- 8 个二进制位：**Byte**（字节），1 Byte = 8 bit
- 2 个字节：**Word**（字）。x86 架构下 1 Word = 2 Byte = 16 bit；RISC-V 中 1 word = 32 bit。

数的机器表示

机器字（machine word）长：计算机进行一次整数运算所能处理的二进制数据的位数，通常也指数据/地址长度。

- **32 位字长**：地址空间 2^{32} B = 4 GB，对内存需求大的应用非常有限。
- **64 位字长**：地址空间约 1.8×10^{19} 字节；目前 x86-64 实际支持 **48 位宽地址**（256 TB）。

机器字在内存中的组织：

- 地址按**字节**定位（字节编址）。
- 机器字的地址 = 其中**第一个字节**的地址。
- 相邻机器字地址相差 4（32 位字）或 8（64 位字）。

字节序（Byte Ordering）

一个机器字内各字节如何排列：

- **大端（Big Endian）**：最低权重字节（LSB）占**高**地址。代表：Sun、PowerPC、网络字节序、Java。
- **小端（Little Endian）**：与 LSB 相反，LSB 占低地址。代表：x86、RISC-V、ARM（默认）。

示例：数值 `0x01234567` 存于地址 `0x100`：

端序	0x100	0x101	0x102	0x103
大端	01	23	45	67
小端	67	45	23	01

实例：`int A = 12345;`（十六进制 0x3039）在 IA32/x86-64（小端）内存中为 `39 30 00 00`；在 Sparc（大端）中为 `00 00 30 39`。`int B = -12345;` 的补码为 `C7 CF FF FF`（小端）。注意 long（8 字节）时小端为 `39 30 00 00 00 00 00 00`，大端符号扩展为 `00 … 30 39`。

网络字节序示例：发送端 x86（小端）本地值 i = 0x12345678，内存布局为 `78 56 34 12`；经 `htonl` 转换后按大端发送字节流 `[12][34][56][78]`；接收端（ARM 设为大端）用 `ntohl` 转回本地序，得到 i = 0x12345678 ✓。若接收端直接 `memcpy` 并按小端解释，会得到 0x78563412 ✕。**字节序本质是“如何解释数据存储顺序”的问题**（磁盘拷贝同理）。

Show Bytes 示例：

```c
typedef unsigned char *byte_pointer;
void show_bytes(byte_pointer start, size_t len) {
  for (int i = 0; i < len; i++)
    printf("%.2x\t", start[i]);
}
void show_int(int x) { show_bytes((byte_pointer)&x, sizeof(int)); }

int main(void) { int x = 0x12345678; show_int(x); } // 输出 78 56 34 12（小端）
```

LSB 在低地址端，MSB（most significant bit）在高地址端。

C 语言基本数据类型大小（字节）

类型	32-bit	x86-32	x86-64

char	1	1	1
short	2	2	2
int	4	4	4
long	4	4	8 (Linux)
long long	8	8	8
float	4	4	4
double	8	8	8
long double	8	10/12	10/16
char * 及其他指针	4	4	8

整数的二进制编码（设字长为 w）

无符号数（原码）：

$$\mathrm{B2U}(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$$

带符号数（补码，Two's Complement）：

$$\mathrm{B2T}(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

其中 x_{w-1} 为**符号位**（补码中 MSB：1 表示负，0 表示非负）。

补码让硬件更简洁：加法与减法在电路层面统一。

示例（short，w=16）：`short x = 12345;` 编码 `30 39`（00110000 00111001）；`short y = -12345;` 编码 `CF C7`（11001111 11000111）。

求补码的规则：

- 非负数：补码 = 原码（反码是原码各位取反）。
- 负数：补码 = 绝对值的反码 + 1 = 2^w + 该负数。
- 一个数的补码：其相反数的补码"按位取反再加一"。

补码为何如此表示：固定位宽 W 的系统中所有运算均是模 2^W 的。

$$A + (\sim A + 1) = A + (2^W - A) = 2^W \equiv 0 (\mathrm{mod}\, 2^W)$$

即 $\sim A + 1$ 是 A 在模 2^W 下的**加法逆元**——这正是"补码"名字的来源。

取值范围

量	值	位模式	说明
$U_{\min}$	0	000…0	
$U_{\max}$	$2^w - 1$	111…1	
$T_{\min}$	-2^{w-1}	100…0	
$T_{\max}$	$2^{w-1} - 1$	011…1	
−1	−1	111…1	全 1 即 −1

w=16 时的具体值：$U_{\max}$ = 65535（FF FF）；$T_{\max}$ = 32767（7F FF）；$T_{\min}$ = −32768（80 00）；−1（FF FF）；0（00 00）。

无符号数与带符号数的转换

二进制位串表示不变，只是解释不同。$w = 4$ 时：位串 0000–0111 两者的值相同（0–7）；1000 起，无符号解释为 8–15，补码解释为 −8 至 −1。

转换公式：

$$ux = \begin{cases} x & x \geq 0 \\ x + 2^w & x < 0 \end{cases}$$

整数加减法：圆环模型

把 w 位整数看作模 2^w 的**圆环**：

- $x + y$：x 顺时针移动 y 个位置（$y > 0$）。
- $x - y$：x 逆时针移动 y 个位置；由于 x 逆时针移 y 位 = 顺时针移 $R - y$ 位（$R = 2^w$），且“减 y”等价于“加全 1 的 w 位串（即 -1 的补码模意义）”，**减法可以用取反加一（加补码）代替**，实现了加减法在电路层面的统一。
- 补码系统的核心：**把负整数映射到无符号数的高数值范围**。如 $T_{\max} + 1 = T_{\min}$，$U_{\max}$（全 1）即 -1，$U_{\max} - 1$ 即 -2。

补码加法公式

$$[x]_{\text{补}} + [y]_{\text{补}} \equiv [x + y]_{\text{补}} \pmod{2^w}$$

意义：负整数用补码表示后可与正整数同样处理，运算器**只需要加法器**。

证明要点：

- $x > 0, y > 0$：正数补码与原码一致，$x + y > 0$，显然成立。
- $x > 0, y < 0$（$x + y > 0$）：$[x]_{\text{补}} = x$，$[y]_{\text{补}} = 2^w + y$，故 $[x]_{\text{补}} + [y]_{\text{补}} = x + y + 2^w \equiv x + y = [x + y]_{\text{补}} \pmod{2^w}$。
- $x > 0, y < 0$（$x + y < 0$）：同理 $x + y + 2^w = [x + y]_{\text{补}}$。

C 语言中的无符号数与带符号数

- **常数**默认是带符号数；有后缀 `U` 则为无符号数（如 `0U`、`4294967259U`）。
- **混合运算时，带符号数默认转换为无符号数**——包括比较操作符。

实例（w=32）：

Constant1	Constant2	比较结果	按何种语义
0	0U	==	unsigned
−1	0	<	signed
−1	0U	>	unsigned
2147483647	−2147483647-1	>	signed
2147483647U	−2147483647-1	<	unsigned
−1	−2	>	signed
(unsigned)-1	−2	>	unsigned
2147483647	2147483648U	<	unsigned
2147483647	(int)2147483648U	>	signed

何时采用无符号数

适用场景：**模运算、按位运算**。建议：**不能仅仅因为取值范围非负就使用无符号数**。

反例一（死循环）：

```
unsigned i;
for (i = cnt-2; i >= 0; i--)  // unsigned 永远 >= 0，循环不终止
  a[i] += a[i+1];
```

反例二：

```c
#define DELTA sizeof(int)   // sizeof 返回 size_t（无符号）
int i;
for (i = CNT; i - DELTA >= 0; i -= DELTA)  // i-DELTA 被转为无符号，恒 >= 0
  ...
```

无符号数加法

操作数位宽 w 位，真实结果 $w+1$ 位，**放弃进位**取 w 位：

$$\mathrm{UAdd}_{w(u,v)} = (u+v) \bmod 2^w = \begin{cases} u+v & u+v < 2^w \\ u+v-2^w & u+v \geq 2^w \quad \text{（溢出）} \end{cases}$$

补码加法及其溢出

补码加法与无符号加法**位级完全一致**（都放弃进位）。C 语言中：

```c
int s, t, u, v;
s = (int)((unsigned)u + (unsigned)v);   // 位级操作
t = u + v;                              // s == t
```

补码加法结果：

$$\mathrm{TAdd}_{w(u,v)} = \begin{cases} u+v+2^w & u+v < \mathrm{TMin}_w \quad \text{(NegOver 向下溢出)} \\ u+v & \mathrm{TMin}_w \leq u+v \leq \mathrm{TMax}_w \\ u+v-2^w & u+v > \mathrm{TMax}_w \quad \text{(PosOver 向上溢出)} \end{cases}$$

即真实和在 $[-2^w, -2^{w-1}-1]$ 区间时加 2^w（负溢出），在 $[2^{w-1}, 2^w-1]$ 区间时减 2^w（正溢出）。

例题（填空）

已知 32 位整数 $X = -101$（十进制）：

- $101 = 0\text{x}65$，取反加一：X 的十六进制补码为 `0xFFFFFF9B`。
- Y 的补码为 `0xFFFFFF6A`，即 $-0\text{x}95 = -149$。
- $X+Y = -250$：补码为 `0xFFFFFF06`。
- $X-Y = -101+149 = 48$：补码为 `0x00000030`。

思考题（3.2 讲解决）：补码加法与无符号加法在位模式层面完全一致，x86 用同一条 `addl` 指令做两种加法——那么 C 层面无符号/带符号的**语义区分**如何完成？答案在条件码与后续指令的不同解释方式（如 `jo` 判溢出、`js` 判符号等）。

整数除以 2^k 与移位

无符号数：`u >> k`（**逻辑右移**，高位补 0）等价于 $\left\lfloor \frac{u}{2^k} \right\rfloor$。

示例（x = 15213 = 0x3B6D）：

表达式	真实除法	计算结果	十六进制
x	15213	15213	3B 6D
x >> 1	7606.5	7606	1D B6
x >> 4	950.8125	950	03 B6
x >> 8	59.4257813	59	00 3B

`unsigned udiv8(unsigned x) { return x/8; }` 编译为 `shrl $3, %eax`（即 `x >> 3`）。

带符号数：`x >> k`（**算术右移**，高位补符号位）等价于 $\left\lfloor \frac{x}{2^k} \right\rfloor$，但当 $x < 0$ 时这是**向下舍入**，而 C 语义要求**向零舍入**，产生舍入错误。

示例（x = −15213 = 0xC493）：

表达式	真实除法	计算结果	十六进制
x	−15213	−15213	C4 93
x >> 1	−7606.5	−7607	E2 49
x >> 4	−950.8125	−951	FC 49
x >> 8	−59.4257813	−60	FF C4

修正：引入偏置量（bias）。想对负数向零舍入（得到 $\left\lceil \frac{x}{2^k} \right\rceil$），可用

$$\left\lfloor \frac{x + 2^k - 1}{2^k} \right\rfloor \quad \Leftrightarrow \quad \text{C: (x + (1<<k) - 1) >> k}$$

- 能被 2^k 整除时：加 $2^k - 1$ 不影响整数部分，偏置量不起作用，结果不变。
- 不能整除时：加 $2^k - 1$ 使整数部分加 1，达到向上（向零）舍入效果。

编译器实现 `int idiv8(int x) { return x/8; }`：

```
testl %eax, %eax    # 判断符号
js    L4            # x < 0 跳转
L3:
sarl $3, %eax       # 算术右移 3 位
ret
L4:
addl $7, %eax       # x += 7 (偏置 2^3-1)
jmp  L3
```

极端情况判断（思考题）

设 x, y 为 32 位带符号整数，ux, uy 为相同位模式的无符号整数，判断下列推断（不成立的给反例）：

推断	结论/分析
`x<0 ⇒ ((x*2)<0)`	不成立：$x = T_{\min}$ 时 $2x$ 正溢出为 0
`ux >= 0`	恒成立
`ux > -1`	恒成立（−1 转无符号为 $U_{\max}$）
`x & 7 == 7 ⇒ ((x<<30) < 0)`	不成立：需注意运算优先级与位被移出的情况
`(x|-x) >> 31 == -1`	不成立：$x = 0$ 或 $x = T_{\min}$ 时 $x \mid -x$ 的符号位不为 1
`x <= 0 ⇒ -x >= 0`	不成立：$x = T_{\min}$ 时 $-x = T_{\min} < 0$
`x >= 0 ⇒ -x <= 0`	成立
`x > y ⇒ -x < -y`	不成立：$y = T_{\min}$ 时 $-y$ 仍为 $T_{\min}$
`x>0 && y>0 ⇒ x+y>0`	不成立：可能正溢出
`ux >> 3 == ux/8`	成立（无符号逻辑右移）
`x >> 3 == x/8`	不成立：$x < 0$ 时右移向下舍入，除法向零舍入
`x & (x-1) != 0`	不成立：x 为 2 的幂（或 0）时按位与为 0

注：以上仅从带符号/无符号数的定义出发，不涉及编译器具体实现；有兴趣可深入 C 语言的“未定义行为”。

2.2 浮点数

引子：一个 C 代码示例

```c
float position = 0.0f, velocity = 0.1f;
for (int i = 0; i < 1000; i++) position += velocity;
printf("position = %.15f\n", position);
printf("position == 100.0f ? %s\n", position == 100.0f ? "true" : "false");
printf("error = %.15f\n", position - 100.0f);
```

结果 `position != 100.0f`：因为 0.1 无法用二进制精确表示，每次累加都引入微小误差，千次累加后误差可见。**浮点数只能精确表示形如 $\frac{X}{2^k}$ 的数**。

二进制小数：$b_i b_{i-1} \ldots b_2 b_1 b_0 . b_{-1} b_{-2} \ldots = \sum_k b_k \cdot 2^k$（$k$ 从 $-j$ 到 i）。例：$\frac{5.3}{4} = 101.11_2$，$\frac{2.7}{8} = 10.111_2$，$\frac{0.63}{64} = 0.111111_2$。

无法精确表示的例子：$\frac{1}{3} = 0.0101[01]\ldots_2$，$\frac{1}{5} = 0.0011[0011]\ldots_2$，$\frac{1}{10} = 0.000110011[0011]\ldots_2$。

IEEE 754 标准的表示形式

数字形式：$(-1)^s \times M \times 2^E$

- s：符号；M：有效数（significand），规格化时位于 $[1.0, 2.0)$；E：指数（阶码）。

位级布局：`s | exp | frac`，exp 域编码 E，frac 域编码小数部分（尾数 mantissa）。

类型	指数域宽度	小数域宽度	总位数
单精度 float	8	23	32
双精度 double	11	52	64
扩展精度 x86	15	63	80（1 比特无用）
半精度 FP16	5	10	16
BF16	8	7	16

三类浮点数

规格化浮点数（Normalized）

条件：exp $\neq$ 000...0 且 exp $\neq$ 111...1。

- 真实阶码 $E = \text{Exp} - \text{Bias}$，其中 Exp 为指数域的无符号值，$\text{Bias} = 2^{e-1} - 1$（$e$ 为 exp 域位数）。单精度 Bias = 127（Exp 取 1…254，$E \in [-126, 127]$）；双精度 Bias = 1023。
- **小数域隐含前导 1**：$M = 1.\text{xxx...x}_2$，因此第一位 1 不必存储。frac = 000…0 时 $M = 1.0$ 最小；frac = 111…1 时 $M = 2.0 - \varepsilon$ 最大（单精度 $\varepsilon = 2^{-23}$）。

示例：float $F = 15213.0$：

$$15213 = 11101101101101_2 = 1.1101101101101_2 \times 2^{13}$$

- $M = 1.1101101101101_2$，frac = `11011011011010000000000`
- $E = 13$，$\text{Exp} = 13 + 127 = 140 = 10001100_2$
- 完整位串：`0 10001100 11011011011010000000000`，十六进制 `0x466DB400`。

非规格化浮点数（Denormalized）

条件：exp = 000...0。

- $E = 1 - \text{Bias}$；$M = 0.\text{xxx...x}$（xxx 即 frac 各位）。
- `frac = 000…0`：表示 0（注意有 +0 与 −0）。
- `frac != 000…0`：表示非常接近 0 的数，精度逐步丧失，称为**逐步下溢（Gradual underflow）**。对比：整数表示“正确”但特定输入下崩溃（溢出），浮点则以精度换平滑。

特殊值

条件：exp = 111...1。

- `frac = 000…0`：**无穷**，用于表示溢出结果，如 $\frac{1.0}{0.0} = +\infty$，$-\frac{1.0}{0.0} = -\infty$。
- `frac != 000…0`：**NaN**（Not-a-Number），如 $\sqrt{-1}$、$\infty - \infty$、$\infty \times 0$。

数轴上的分布

从左到右依次为：$-\infty$、负规格化数、负非规格化数、$-\frac{0}{+}0$、正非规格化数、正规格化数、$+\infty$；NaN 在两端之外。

一种“小”浮点数实例（E4M3）

8 位浮点数：指数域 4 位、小数域 3 位，其余规则符合 IEEE 754（规格化/非规格化、0、inf、NaN）。偏置量 $= 2^4 - 1 - 1 = 7$。FP8（2022–2023 年提出）用于加速深度学习训练/推理，含 E4M3 与 E5M2 两种编码（此处 M 表示 Mantissa）。

exp 域与 E、2^E 的对应（E4M3）：

Exp	exp	E	2^E	备注
0	0000	−6	1/64	非规格化
1	0001	−6	1/64	
2	0010	−5	1/32	
3	0011	−4	1/16	
4	0100	−3	1/8	
5	0101	−2	1/4	
6	0110	−1	1/2	
7	0111	0	1	
8	1000	+1	2	
9	1001	+2	4	
10	1010	+3	8	
11	1011	+4	16	
12	1100	+5	32	
13	1101	+6	64	
14	1110	+7	128	
15	1111	n/a	—	inf, NaN

AI 中的浮点格式一览：

格式	指数	小数	总位数	使用场景
FP32	8	23	32	训练默认，精度高
FP16	5	10	16	推理加速，范围小
BF16	8	7	16	训练替代 FP32，范围同 FP32
FP8 (E4M3)	4	3	8	推理，精度优先
FP8 (E5M2)	5	2	8	训练，范围优先

取值范围与特例（单/双精度）

描述	exp	frac	数值
零	00…00	00…00	0.0
最小正非规格化	00…00	00…01	$2^{-\{23,52\}} \times 2^{-\{126,1022\}}$；单 $\approx 1.4 \times 10^{-45}$，双 $\approx 4.9 \times 10^{-324}$
最大非规格化	00…00	11…11	$(1.0 - \varepsilon) \times 2^{-\{126,1022\}}$；单 $\approx 1.18 \times 10^{-38}$，双 $\approx 2.2 \times 10^{-308}$
最小正规格化	00…01	00…00	$1.0 \times 2^{-\{126,1022\}}$（略大于最大非规格化数）
1.0	01…11	00…00	1.0
最大规格化	11…10	11…11	$(2.0 - \varepsilon) \times 2^{\{127,1023\}}$；单 $\approx 3.4 \times 10^{38}$，双 $\approx 1.8 \times 10^{308}$

编码特性：除符号位需先比较、需注意 ±0 与 NaN 外，浮点数（几乎）可以直接用**无符号整数比较**的方式比较大小（不考虑符号位时 NaN 的位模式比其他值都大；实际比较中 NaN 与任何值的比较结果均为“无序”）。

8 位（Bias=7）编码连续性示例：0000 000（0）→ 0000 001（$\frac{1}{512}$，最接近 0）→ … → 0000 111（$\frac{7}{512}$，最大非规格化）→ 0001 000（$\frac{8}{512}$，最小规格化）→ … → 0110 111（$\frac{15}{16}$，最接近 1 的小于侧）→ 0111 000（1）→ 0111 001（$\frac{9}{8}$）→ … → 1110 111（240，最大规格化）→ 1111 000（inf）。注意**非规格化到规格化的平滑过渡**。

Rounding（舍入）

给定实数求浮点表示的流程：先算精确值，再转换为所需精度（可能溢出、可能需要舍入）。

四种舍入模式（例：保留整数元）：

模式	1.40	1.60	1.50	2.50	−1.50
向 0 舍入	1	1	1	2	−1
向下舍入	1	1	1	2	−2
向上舍入	2	2	2	3	−1
向偶数舍入	1	2	2	2	−2

向偶数舍入（Round-To-Even）：计算机内**默认**方式，也称“向最接近值舍入”；其它方式会产生统计偏差。关键规则：恰在两个可能结果**正中间**时，舍入到最低有效数字为**偶数**的那个。

例（保留百分位）：

- 1.2349999 → 1.23（不到一半，向下）
- 1.2350001 → 1.24（大于一半，向上）
- 1.2350000 → 1.24（恰好一半，向上使末位为偶数 4）
- 1.2450000 → 1.24（恰好一半，向下使末位为偶数 4）

二进制情形：“even” 意味着最低有效位为 0；只有当最低有效位右侧位串恰为 `100…`（即正好一半）时才需要决策。

例（舍入到小数点后 2 位）：

数值	二进制	舍入后	操作	结果
$2\frac{3}{32}$	10.00011	10.00	($<\frac{1}{2}$，向下)	2
$2\frac{3}{16}$	10.00110	10.01	($>\frac{1}{2}$，向上)	$2\frac{1}{4}$
$2\frac{7}{8}$	10.11100	11.00	($=\frac{1}{2}$，向上，末位凑偶)	3
$2\frac{5}{8}$	10.10100	10.10	($=\frac{1}{2}$，向下，末位已偶)	$2\frac{1}{2}$

实数 → 浮点数三步法

1. **规格化**：写成 1.xxx × 2^E 形式（前导 1）。
2. **向偶数舍入**：使有效数符合 frac 域位数需求。
3. **调整（Postnormalize）**：舍入可能引起进位溢出（如 1.111 进位为 10.000），需右移一位并使 E 加 1。

例：将 8 位无符号数转为 8 位浮点（exp 4 位，frac 3 位，Bias = 7）：

数值	二进制	规格化 $M \times 2^E$	舍入后	调整后结果
128	10000000	1.0000000 × 2^7	1.000，不进位	128
15	00001111	1.1110000 × 2^3	1.111，不进位	15
17	00010001	1.0001000 × 2^4	1.000，不进位	16（向下舍入丢失 $\frac{1}{16}$）
19	00010011	1.0011000 × 2^4	1.010，进位	20
138	10001010	1.0001010 × 2^7	1.001，进位	144
63	00111111	1.1111100 × 2^5	进位为 10.000	64（postnormalize：1.000 × 2^6）

C 语言中的浮点数

类型转换基本原则（int 为 32 位时）：

- **double/float → int**：尾数部分**截断**（向零舍入）；溢出或 NaN 时结果**未定义**（通常置为 $T_{\min}$ 或 $T_{\max}$）。
- **int → double**：能够**精确**转换（52 位 frac 足够容纳 32 位整数）。
- **int → float**：**不会溢出，但可能被舍入**（frac 只有 23 位）。

Floating Point Puzzles（判断是否恒成立，d, f 均非 NaN）：

表达式	结论
`x == (int)(float) x`	不成立：int → float 可能舍入，如 x = TMax
`x == (int)(double) x`	成立：int → double 精确，double → int 截断回原值
`f == (float)(double) f`	成立：float → double 精确，转回无损
`d == (float) d`	不成立：double → float 可能舍入（也可能溢出为 inf）
`f == -(-f)`	成立：取负只翻符号位
`2/3 == 2/3.0`	不成立：左边整数除法得 0，右边为 0.66…
`d < 0.0 ⇒ (d*2) < 0.0`	成立：乘 2 只加指数，不会变号
`d > f ⇒ -f > -d`	成立
`d * d >= 0.0`	成立：可能溢出为 $+\infty$（仍 ≥ 0）
`(d+f)-d == f`	不成立：大数吞小数，如 $d = 1 \times 10^{10}, f = 1.0$ 时 f 被吞没

浮点加法不满足结合律示例：

```
float arr[] = {1e10f, 1.0f, -1e10f};
float sum1 = (1e10f + 1.0f) + (-1e10f);    // 0.0, 1.0 被大数吞没
float sum2 = 1.0f + (-1e10f) + 1e10f;      // 0.0
float sum3 = (-1e10f) + 1e10f + 1.0f;      // 1.0 ✓ 改变结合顺序可修正
```

结论：浮点运算中**顺序很重要**；这也是编译器不能随意对浮点表达式重新结合的原因。

3.1 80x86 汇编语言与 C 语言（一）

本讲要点

程序的语义等价转换：C → 汇编，包括但不限于：

- 基本计算、控制流、循环、函数调用与返回、数组/结构等复合数据类型访问。
- **（部分）指令/函数/数据的“定位”**：局部变量（如 `-20(%rbp)`）、（条件/无条件）跳转指令的目的、静态函数入口地址、函数返回地址、函数的参数与返回值（`%eax/%rax`）。
- 代码优化。

示例（`if/else` → 汇编）：

```
if (val >= 0) return val; else return -1;
```

```
cmpl $0, -20(%rbp)
js   .L4            # 符号位为 1（负）则跳转
movl -20(%rbp), %eax
jmp  .L5
.L4:
movl $-1, %eax
.L5:
```

汇编程序员眼中的体系结构

冯·诺依曼架构，又称**控制流（control flow）架构**。关键部件：

- **程序计数器（PC）**：存储下一条指令的地址；x86-64 中名为 **RIP**。
- **寄存器堆（register file）**：处理器内部以名字访问的快速存储单元。
- **条件码（condition codes）**：存储最近执行指令的结果状态信息，用于条件指令的判断执行。
- **内存（main memory）**：以字节编码的连续存储空间，存放用户程序代码/数据、操作系统代码/数据，以及栈（Stack）与堆（Heap）。

寄存器与内存的比较：

项目	寄存器	内存
位置	CPU 内部	CPU 外部
访问速度	快	慢
容量	小	大
成本	高	低
表示方式	用名字表示	用地址表示
地址	没有	可用多种寻址方式形成

存储层次（补充）：L1（约 3–4 cycles）→ L2（约 10–28 cycles）→ L3（约 40–120 cycles）→ 内存 DRAM（约 100+ cycles）。示例数据：Intel Nehalem Core i7：L1 = 4，L2 = 10，L3 ≈ 40；Apple M4：L1 ≈ 3–4，L2 ≈ 18–28，SLC ≈ 90–120（Apple 无传统 L3，用 System Level Cache）；AMD Zen5 Ryzen 9000：4/14/46–50；Intel Lion Cove（引入 L0+L1 分层）：L0 = 4，L1 = 9，L2 = 17，L3 ≈ 80+。

从 C 生成汇编

```
long plus(long x, long y);
void sumstore(long x, long y, long *dest) {
  long t = plus(x, y);
  *dest = t;
}
```

```
sumstore:
  pushq %rbx
  movq  %rdx, %rbx      # 保存 dest
  call  plus            # 返回值在 %rax
  movq  %rax, (%rbx)    # *dest = t
  popq  %rbx
  ret
```

生成命令：`gcc -Og -S sum.c`（`-S` 生成 sum.s；`-c` 生成目标文件 sum.o，内含机器指令）。

汇编语言的基本特点

数据类型：

- 整型：宽度 1、2、4 或 8 字节；浮点：4、8 或 10 字节。
- **无复杂数据类型**：结构/数组就是内存中连续分配的字节，由编译器解释。
- 汇编级的数据既可表示**数值**，也可表示**内存地址**。

操作类型：

- 对寄存器或内存数据进行算术/逻辑操作。
- 数据传送：内存与寄存器之间、寄存器与寄存器之间。
- 控制转移（transfer control）：无条件跳转、函数调用及返回、条件跳转。

数据格式后缀（AT&T 汇编用指令后缀区分操作数大小；x86 中“字”指 16 位）：

后缀	类型	大小
B	BYTE	1 字节（8 位）
W	WORD	2 字节（16 位）
L	LONG	4 字节（32 位）
Q	QUADWORD	8 字节（64 位）

第一条指令实例与反汇编

`*dest = t;` 即“把 t 存到 dest 指定的内存地址”，对应：

- `movq %rax, (%rbx)`：将 8 字节的值（%rax，即 t）存入 M[%rbx]（即 *dest）。
- 目标代码：`0x40059e: 48 89 03`——三字节指令，指令自身地址 0x40059e。

反汇编（`objdump -d sum.o`）：

```
0000000000400595 <sumstore>:
  400595: 53              push   %rbx
  400596: 48 89 d3        mov    %rdx,%rbx
  400599: e8 f2 ff ff ff  callq  400590 <plus>
  40059e: 48 89 03        mov    %rax,(%rbx)
  4005a1: 5b              pop    %rbx
  4005a2: c3              retq
```

gdb 方式：`disassemble sumstore` 反汇编某函数；`x/14xb sumstore` 显示从 sumstore 开始的 14 个字节（格式 `x/nfu <addr>`）。

x86-64 通用寄存器

16 个 64 位通用寄存器（括号内为低 32 位别名）：%rax（%eax）、%rdx（%edx）、%rcx（%ecx）、%rbx（%ebx）、%rsi（%esi）、%rdi（%edi）、%rsp（%esp）、%rbp（%ebp）、%r8–%r15（%r8d–%r15d）。

x86-32（后向兼容）的历史用途（基本已过时）：%eax = accumulator，%ecx = counter，%edx = data，%ebx = base，%esi = source index，%edi = destination index，%esp = stack pointer，%ebp = base pointer；另有 %ax、%ah、%al 等 16/8 位虚拟寄存器。

数据传送指令 mov

格式：`movq Source, Dest`（**源在左，目的在右**，AT&T 风格）。

操作数类型：

- **立即数（Imm）**：整型常数，前缀 `$`，如 `$0x400`、`$-533`；位宽 1/2/4/8 字节。
- **寄存器（Reg）**：16 个通用寄存器之一（%rsp 有特殊用途）。
- **内存（Mem）**：8 个连续字节（对 q 而言），起始地址由地址表达式指定，如 `(%rax)`。

组合规则：Imm→Reg/Mem、Reg→Reg/Mem、Mem→Reg 均**允许**；**两个操作数不能都为内存**。

类型	示例	C 语言表示
Imm→Reg	`movq $0x4, %rax`	`temp = 0x4;`
Imm→Mem	`movq $-147, (%rax)`	`*p = -147;`
Reg→Reg	`movq %rax, %rdx`	`temp2 = temp1;`
Reg→Mem	`movq %rax, (%rdx)`	`*p = temp;`
Mem→Reg	`movq (%rax), %rdx`	`temp = *p;`

寻址模式

简单寻址：

- 间接寻址 `(R)`：地址 = Mem[Reg[R]]，如 `movq (%rcx), %rax`。
- 基址+偏移 `D(R)`：地址 = Mem[Reg[R]+D]，如 `movq 8(%rbp), %rdx`。

变址模式：常见形式 `D(Rb, Ri, S)`，地址 = Mem[Reg[Rb] + S×Reg[Ri] + D]，其中 D 为常量偏移，Rb 为基址寄存器（16 个之一），S 为比例因子（1、2、4 或 8），Ri 为索引寄存器（**%rsp 不可作索引寄存器**）。

变形：

形式	地址
`(Rb,Ri)`	Mem[Reg[Rb]+Reg[Ri]]
`D(Rb,Ri)`	Mem[Reg[Rb]+Reg[Ri]+D]
`(Rb,Ri,S)`	Mem[Reg[Rb]+S×Reg[Ri]]

寻址实例（%rdx = 0xf000，%rcx = 0x0100）：

地址表达式	地址计算	结果
`0x8(%rdx)`	0xf000 + 0x8	0xf008
`(%rdx,%rcx)`	0xf000 + 0x100	0xf100
`(%rdx,%rcx,4)`	0xf000 + 4×0x100	0xf400
`0x80(,%rdx,2)`	2×0xf000 + 0x80	0x1e080

参数传递与 swap 示例

参数通过寄存器传递：前 6 个参数依次放入 **%rdi, %rsi, %rdx, %rcx, %r8, %r9**；第 7 个及以上的参数依次**从右向左**放入栈中（前 6 个方式不变）。

```
void swap(long *xp, long *yp) {
  long t0 = *xp;
  long t1 = *yp;
  *xp = t1;
  *yp = t0;
}
```

```
swap:
  movq (%rdi), %rax   # t0 = *xp
```

```
  movq (%rsi), %rdx   # t1 = *yp
  movq %rdx, (%rdi)   # *xp = t1
  movq %rax, (%rsi)   # *yp = t0
  ret
```

执行过程分解（设 xp = 0x120 指向 123，yp = 0x100 指向 456）：

步骤	%rax	内存 0x120	内存 0x100
初始	—	123	456
`movq (%rdi),%rax`	123	123	456
`movq (%rsi),%rdx`	123	123	456（此时 %rdx = 456）
`movq %rdx,(%rdi)`	123	456	456
`movq %rax,(%rsi)`	123	456	123

若交换的是 `int`（32 位数据）：使用 **%eax、%edx 与 `movl` 指令**——指针本身仍是 64 位（仍在 %rdi/%rsi），但被操作的数据是 32 位。

地址计算指令 lea

`leaq Src, Dest`：Src 是地址计算表达式，**计算出的值（地址）直接赋给 Dest，无需访存**。

两大用途：

- 地址计算，如 `p = &x[i];`
- 进行 $x + ky$ 型整数计算（$k \in \{1, 2, 4, 8\}$）。

示例：

```
long m12(long x) { return x * 12; }
```

```
leaq (%rdi,%rdi,2), %rax   # t = x + x*2 = 3x
salq $2, %rax              # return t << 2 = 12x
```

整数算术逻辑指令

双操作数（格式 `op Src, Dest`，即 Dest = Dest op Src）：

指令	操作
`addq Src,Dest`	Dest ← Dest + Src
`subq Src,Dest`	Dest ← Dest − Src
`imulq Src,Dest`	Dest ← Dest × Src（取低 64 位）
`salq Src,Dest`	Dest ← Dest << Src（与 shlq 等价）
`sarq Src,Dest`	Dest ← Dest >> Src（算术右移）
`shrq Src,Dest`	Dest ← Dest >> Src（逻辑右移）
`xorq Src,Dest`	Dest ← Dest ^ Src
`andq Src,Dest`	Dest ← Dest & Src
`orq Src,Dest`	Dest ← Dest | Src

单操作数：`incq Dest`（+1）、`decq Dest`（−1）、`negq Dest`（取负）、`notq Dest`（按位取反）。

综合示例：arith

```
long arith(long x, long y, long z) {
  long t1 = x + y;
  long t2 = z + t1;
```

```
  long t3 = x + 4;
  long t4 = y * 48;
  long t5 = t3 + t4;
  long rval = t2 * t5;
  return rval;
}
```

```
arith:
  leaq  (%rdi,%rsi), %rax    # t1 = x+y
  addq  %rdx, %rax           # t2 = z+t1 → %rax
  leaq  (%rsi,%rsi,2), %rdx  # y*3
  salq  $4, %rdx             # t4 = y*48
  leaq  4(%rdi,%rdx), %rcx   # t5 = x+4+t4
  imulq %rcx, %rax           # rval = t2*t5
  ret
```

寄存器用途：%rdi = x，%rsi = y，%rdx = z（后存 t4），%rax = t1/t2/rval，%rcx = t5。注意 **t3 = x+4 被 `leaq 4(%rdi,%rdx)` 一条指令吸收**（编译器优化：t3 不单独出现）；`leaq` 一条指令完成"加法+比例乘法"，`salq` 完成移位乘，`imulq` 只用了一次。

例题（填空）

```
long arith2(long x, long y, long z) {
  long t1 = x + z + y;
  long t3 = x + 4;
  long t4 = y * 48;
  long t5 = t3 + t4;
  long rval = t1 * t5;
  return rval;
}
```

```
arith2:
  leaq (%rdi,%rdx), %rax        # t1 = x+z
  addq %rsi, %rax               # [填空1] → t1 = x+z+y
  leaq (%rsi,%rsi,3), %rdx      # [填空2] → 4y
  salq $4, %rdx                 # ×16 → t4 = y*48
  leaq 4(%rdi,%rdx), %rcx       # [填空3] → t5 = x+4+t4
  imulq %rdx, %rax
  ret
```

答案：填空 1 = `%rsi`；填空 2 = `3`（$(1+3)\times 16 = 64$，即 $4y \times 16 = 48y$）；填空 3 = `%rcx`。

位运算示例

```
int logical(int x, int y) {
  int t1 = x ^ y;
  int t2 = t1 >> 17;          // 算术右移
  int mask = (1 << 13) - 7;   // 8192 - 7 = 8185
  int rval = t2 & mask;
  return rval;
}
```

```
logical:
  movl %edi, %eax
  xorl %esi, %eax       # t1 = x^y
  sarl $17, %eax        # t2 = t1 >> 17
```

```
    andl $8185, %eax      # rval = t2 & 8185
    ret
```

目前为止要掌握的“定位”问题

- 局部变量的定位（如 `-20(%rbp)`；学过运行栈后会有补充——并不意味着只能如此）。
- （条件/无条件）跳转指令的目的寻址。
- 静态函数入口地址、函数返回地址。
- 函数的参数（%rdi、%rsi、%rdx、%rcx、%r8、%r9）、返回值（%eax/%rax）。

归纳：x86-64 指令的特点

- 支持多种类型的指令操作数：立即数、寄存器、内存数据；**算逻指令可以以内存数据为操作数**。
- 支持多种内存地址计算模式：`Rb + S×Ri + D`，也可用于整数计算（如 `leal/leaq`）。
- **变长指令**：长度 1 ~ 15 字节。